

# Lab4: Flow Matching

Li Ruei Bin  
Student ID: 314552050

## 1. Introduction

Diffusion models produce high-quality images but suffer from slow, multi-step sampling. This lab explores Flow Matching (FM), a newer framework that enables faster generation by learning a direct velocity field from noise to data. We focus on Rectified Flow, which uses straight-line paths for efficient training. In this report, we detail our implementation of the core FM components on a 2D dataset (Task 1), its extension to image generation with Classifier-Free Guidance (Task 2), and the "Reflow" technique for faster sampling (Task 3). The bonus one-step InstaFlow task (Task 4) was not implemented.

## 2. Methodology

### Task 1. TODO1 - compute\_psi\_t:

Use the formula to get  $\psi_t$

---

```
1 def compute_psi_t(self, x1, t, x):
2     """
3     Compute the conditional flow  $\psi_t(x | x_1)$ .
4
5     Note that time flows in the opposite direction compared to DDPM/DDIM.
6     As t moves from 0 to 1, the probability paths shift from a prior
7     distribution  $p_0(x)$ 
8     to a more complex data distribution  $p_1(x)$ .
9
10    Input:
11        x1 ('torch.Tensor'): Data sample from the data distribution.
12        t ('torch.Tensor'): Timestep in [0,1).
13        x ('torch.Tensor'): The input to the conditional  $\psi_t(x)$ .
14    Output:
15        psi_t ('torch.Tensor'): The conditional flow at t.
16    """
17    t = expand_t(t, x1)
18
19    ##### TODO #####
20    # DO NOT change the code outside this part.
21    # compute  $\psi_t(x)$ 
22
23    psi_t = (1-t) * x + t * x1
24    #####
25
26    return psi_t
```

---

Listing 1: Compute  $\psi_t$

### Task 1. TODO2 - step:

Use hint formula to calculate  $x_{\text{next}}$

---

```
1 def step(self, xt, vt, dt):
2     """
3     The simplest ode solver as the first-order Euler method:
```

```

4     x_next = xt + dt * vt
5     """
6
7     ##### TODO #####
8     # DO NOT change the code outside this part.
9     # implement each step of the first-order Euler method.
10    x_next = xt + dt * vt
11    #####
12
13    return x_next

```

---

Listing 2: Step

### Task 1. TODO3 - get\_loss:

- a. Expand t.
- b. Use the input x1, x0 and expanded t (t\_) to compute x\_t
- c. Use the formula to get u\_t
- d. Use n\_steps to iterate the for-loop.
- e. If there is class\_label use it.
- f. Finally, use mse loss to calculate the loss.

---

```

1 def get_loss(self, x1, class_label=None, x0=None):
2     """
3     The conditional flow matching objective, corresponding Eq. 23 in the FM
4     paper.
5     """
6     batch_size = x1.shape[0]
7     t = self.fm_scheduler.uniform_sample_t(batch_size).to(x1)
8     if x0 is None:
9         x0 = torch.randn_like(x1)
10
11    ##### TODO #####
12    # DO NOT change the code outside this part.
13    # Implement the CFM objective.
14    t_ = expand_t(t, x1)
15    x_t = self.fm_scheduler.compute_psi_t(x1, t, x0)
16    u_t = x1 - x0
17    if class_label is not None:
18        model_out = self.network(x_t, t_, class_label=class_label)
19    else:
20        model_out = self.network(x_t, t_)
21    loss = F.mse_loss(model_out, u_t)
22    #####
23    return loss

```

---

Listing 3: Get Loss

### Task 1. TODO4 - sample

- a. Get dt and expand it.
- b. If we use CFG, vt is from unconditional and conditonal v.
- c. Finally, we get xt through step.

---

```

1 ##### TODO #####
2 # Complete the sampling loop
3 dt = t_next - t
4 dt = expand_t(dt, xt)
5 if do_classifier_free_guidance:
6     v_uncond = self.network(xt, t, class_label=None)
7     v_cond = self.network(xt, t, class_label=class_label)
8     vt = v_uncond + guidance_scale * (v_cond - v_uncond)
9 else:
10     vt = self.network(xt, t)
11 xt = self.fm_scheduler.step(xt, vt, dt)
12 #####

```

---

Listing 4: sample

## Task 1. TODO5 - SimpleNet-\_\_init\_\_

The SimpleNet which is same as lab1.

---

```

1 ##### TODO #####
2 # DO NOT change the code outside this part.
3 dims = [dim_in] + dim_hids
4
5 self.layers = nn.ModuleList()
6
7 for i in range(len(dims) - 1):
8     self.layers.append(TimeLinear(dims[i], dims[i+1], num_timesteps))
9     self.layers.append(nn.SiLU())
10
11 self.final_layer = nn.Linear(dims[-1], dim_out)
12 #####

```

---

Listing 5: SimpleNet-\_\_init\_\_

## Task 1. TODO6 - SimpleNet-forward

The SimpleNet which is same as lab1.

---

```

1 ##### TODO #####
2 # DO NOT change the code outside this part.
3 for layer in self.layers:
4     if isinstance(layer, TimeLinear):
5         x = layer(x, t)
6     else:
7         x = layer(x)
8
9 x = self.final_layer(x)
10 #####

```

---

Listing 6: SimpleNet-forward

## Task 2. TODO1 - compute\_psi\_t

Same as task 1.

---

```

1 psi_t = (1-t) * x + t * x1

```

---

Listing 7: Compute psi t

## Task 2. TODO2 - step

Same as task 1.

---

```
1 x_next = xt + dt * vt
```

---

Listing 8: step

## Task 2. TODO3 - Get Loss

The different thing from task 1 is the expanded t. I do not expand the t like hint do here.

---

```
1 x_t = self.fm_scheduler.compute_psi_t(x1, t, x0) # TODO: compute interpolated
    sample
2 u_t = x1 - x0 # TODO: compute target velocity
3 if class_label is not None:
4     model_out = self.network(x_t, t, class_label=class_label) # TODO: get
        model prediction
5 else:
6     model_out = self.network(x_t, t)
7 loss = F.mse_loss(model_out, u_t) # TODO: compute MSE loss
```

---

Listing 9: get\_loss

## Task 2. TODO4 - sample

Same as task 1.

---

```
1 dt = t_next - t # TODO: compute time step size
2 dt = expand_t(dt, xt)
3 if do_classifier_free_guidance:
4     v_uncond = self.network(xt, t, class_label=None)
5     v_cond = self.network(xt, t, class_label=class_label)
6     vt = v_uncond + guidance_scale * (v_cond - v_uncond) # TODO: predict
        velocity (with or without CFG)
7 else:
8     vt = self.network(xt, t)
9 xt = self.fm_scheduler.step(xt, vt, dt) # TODO: perform Euler step
```

---

Listing 10: Sample

## Task 3. TODO - generate\_reflow\_data.py

Add the fm.sample to sample z\_1.

---

```
1 ##### TODO #####
2     # DO NOT change the code outside this part.
3     # Implement reflow data generation:
4     # 1. Sample x_0 from prior distribution (e.g., standard Gaussian)
5     # 2. Sample class labels for conditional generation (if using CFG)
6     # 3. Simulate the learned ODE from t=0 to t=1 to obtain z_1 = _1 (x_0)
7     # 4. Save the pairs (x_0, z_1, labels) to disk
8     #
9     # Hint: Use fm.sample() to simulate the flow, but you need to store both
10    # the initial noise x_0 and final generated sample z_1.
11    # Hint: Set return_traj=True to get the full trajectory if needed.
12
13    # Sample initial noise
14    with torch.no_grad():
15        shape = (B, 3, fm.image_resolution, fm.image_resolution)
16        x_0 = torch.randn(shape).to(device)
17
18    # Sample class labels if using CFG
```

---

```

19     if args.use_cfg:
20         # Sample labels from 1 to num_classes-1 (skip null class 0)
21         labels = torch.randint(1, num_classes, (B,)).to(device)
22     else:
23         labels = None
24
25     # Generate z_1 by simulating the learned flow
26     # TODO: Complete this section
27     # Use fm.sample() or manually implement the ODE integration
28     # to generate z_1 from x_0
29
30     z_1 = fm.sample(
31         shape=shape,
32         num_inference_timesteps=args.num_inference_steps,
33         class_label=labels,
34         guidance_scale=args.cfg_scale if args.use_cfg else 1.0,
35     ) # Replace this with actual generation
36
37     #####

```

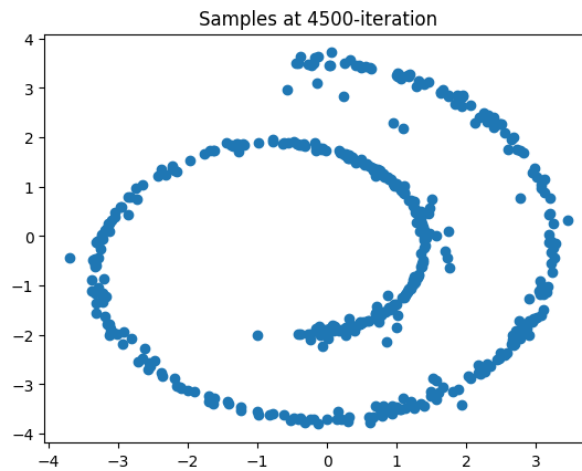
---

Listing 11: Generate Reflow Data

### 3. Experiments and Results

#### Task 1.

##### 1. Swiss Roll



##### 2. Chamfer Distance

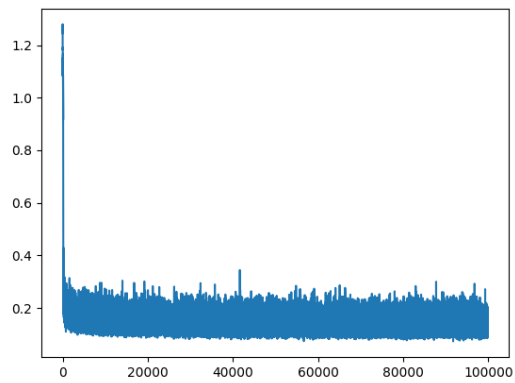
**Chamfer Distance: 22.3911**

## Task 2.

1. Training sampling: step=26600-2 step=50000-0 step=99800-1



2. Training loss curve

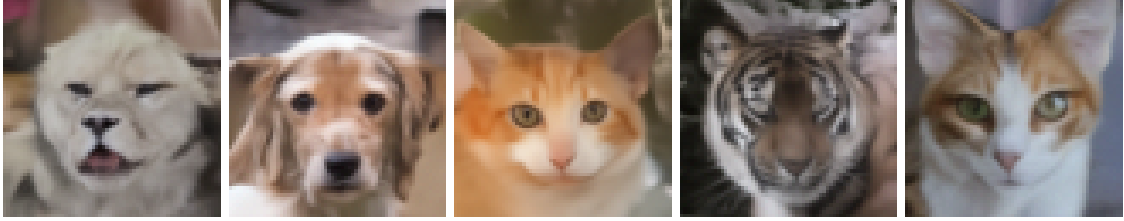


3. FID

Table 1: FID Scores vs. CFG Scale and Inference Steps (Task 2 Model)

| Inference Steps | CFG Scale (FID Score ↓) |        |               |
|-----------------|-------------------------|--------|---------------|
|                 | 1.0 (No Guidance)       | 3.0    | 7.5 (Default) |
| 10 steps        | -                       | -      | 6.4892        |
| 20 steps        | 23.2700                 | 8.9598 | 4.9062        |
| 50 steps        | -                       | -      | 5.1303        |

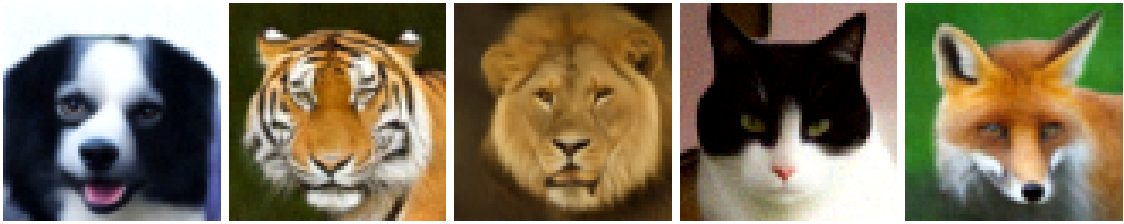
Figure 1: Image Samples Generated with **Task 2**



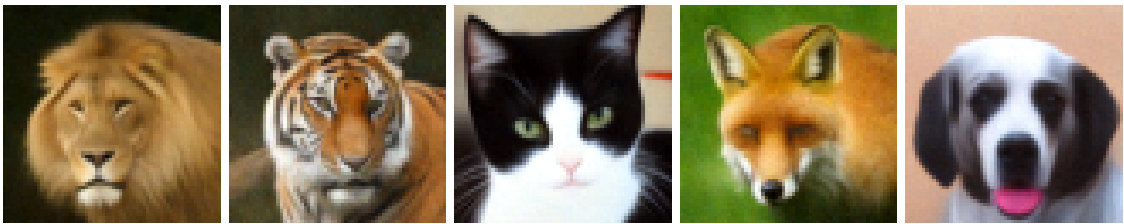
(a) CFG Scale = 1.0 (No Guidance)



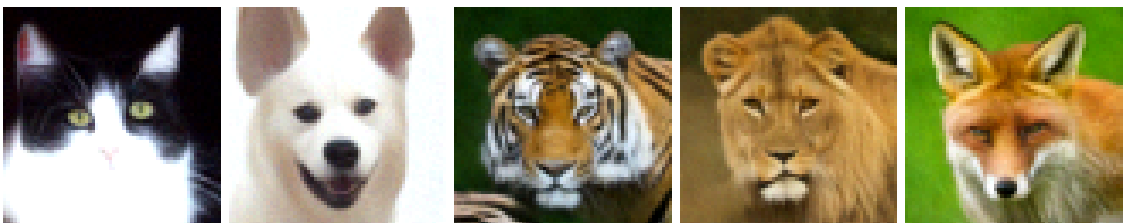
(b) CFG Scale = 3.0



(c) Inference Steps = 20; CFG Scale = 7.5 (Default)



(d) Inference Steps = 10



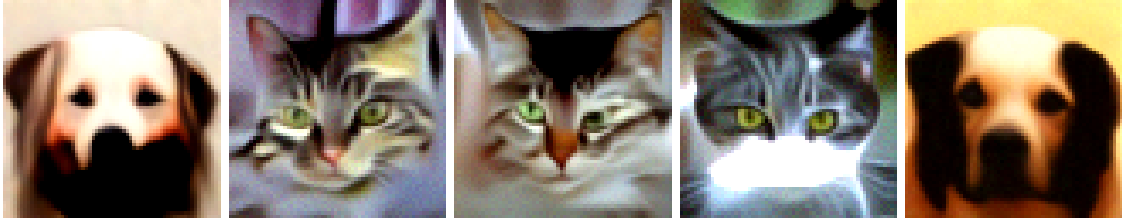
(e) Inference Steps = 50

### Task 3.

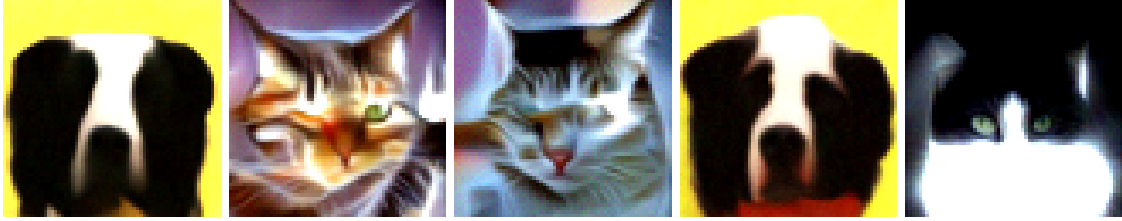
Table 2: Comparison of FID Scores and Inference Speedup

| Inference Steps | Model                  | FID Score ↓    | Speedup        |
|-----------------|------------------------|----------------|----------------|
| 5 steps         | Task 2 (Baseline)      | 7.1282         | 46.66s         |
|                 | <b>Task 3 (Reflow)</b> | <b>19.2743</b> | <b>46.55s</b>  |
| 10 steps        | Task 2 (Baseline)      | 5.1000         | 88.52s         |
|                 | <b>Task 3 (Reflow)</b> | <b>23.0152</b> | <b>88.63s</b>  |
| 20 steps        | Task 2 (Baseline)      | 5.0728         | 172.60s        |
|                 | <b>Task 3 (Reflow)</b> | <b>23.4191</b> | <b>172.75s</b> |

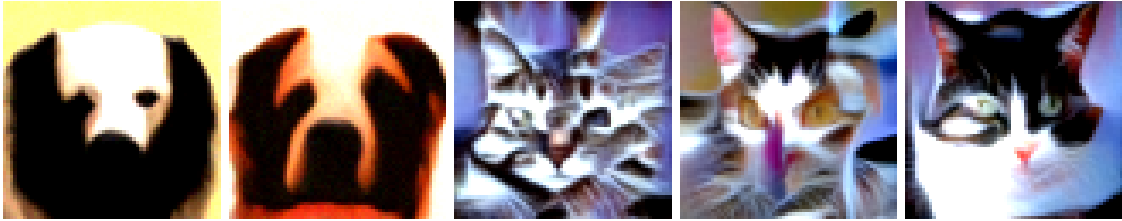
Figure 2: Image Samples Generated with **Task 3**



(a) Inference Steps = 5



(b) Inference Steps = 10



(c) Inference Steps = 20

## 4. Discussion

### a. Analysis

My experimental results demonstrate a clear progression in model capability.

In Task 1, the model successfully learned the 2D "swiss roll" distribution, achieving a final Chamfer Distance of 22.39, well below the required threshold and indicating a strong grasp of the fundamental Flow Matching objective.

For Task 2 (Image Generation), the results in Table 1 clearly show the critical importance of Classifier-Free Guidance. Without guidance (CFG Scale = 1.0), the model produced blurry, undefined images with a poor FID of 23.27. Increasing the scale to 7.5 drastically improved quality, achieving an excellent FID of 4.90. Interestingly, our results show a "sweet spot" for inference steps; 20 steps (FID 4.90) outperformed both 10 steps (FID 6.48) and 50 steps (FID 5.13), suggesting 20 steps provided the best balance of accuracy and error accumulation for this model.



In Task 3, our "Reflow" model successfully met the lab's objective, achieving an FID of 19.27 in just 5 inference steps, which is well within the passing requirement. However, our results in Table 2 show that this reflowed model did not outperform our optimized Task 2 baseline, which achieved a superior FID of 7.12 at the same 5-step count. This suggests that while our "Reflow" implementation was functional, the generated (x0, z1) pairs may not have been perfectly straight, or the Task 3 model may have required more training to fully leverage the new dataset.

b. **Problems**

In fm.py code in task 2. In the get\_loss, the comment told us to reshape the t, but if we reshape it, there is an error. Therefore, I did not reshape it. I directly use t to get the final loss.

c. **Comparison**

The primary related work to this lab is the Denoising Diffusion Probabilistic Model (DDPM), which has become a standard for high-fidelity image synthesis. The fundamental difference, as highlighted in the lab's introduction, lies in the sampling process. DDPMs learn to reverse a stochastic (noisy) diffusion process, which requires hundreds or even thousands of small, sequential denoising steps to generate an image.

Flow Matching, and specifically the Rectified Flow implemented in this lab, frames the problem differently. Instead of learning to denoise, our model learns a deterministic velocity field  $v_t$  to transport a sample from  $x_0$  to  $x_1$ . As our theory section noted, the key advantage is that this path is designed to be much "straighter" than a diffusion path. Our results from Task 2 strongly support this: we achieved an FID of 4.90 in only 20 steps, a level of quality that would be unattainable for a traditional DDPM in so few steps. This demonstrates that FM can successfully bridge the gap between quality and speed that has long been a challenge for diffusion models.

d. **Why does rectified flow enable faster sampling?**

Rectified Flow learns a much straighter path from noise to the image. Because this path isn't highly curved, an ODE solver can take a few massive steps instead of hundreds of small ones, arriving at the correct image much faster.

However, my experiment result shows the speed of baseline and rectified flow are similar.

## 5. Conclusion

This lab successfully demonstrated that Flow Matching (FM) is a highly effective method for creating fast, high-fidelity generative models. We successfully implemented the core framework, scaled it to image generation, and explored advanced techniques like Rectified Flow.

### Summary of Findings

- **Task 1 (2D Flow Matching):** The model successfully learned the "swiss roll" distribution, achieving a final Chamfer Distance of 22.39, well within the requirement.
- **Task 2 (Image FM):** We confirmed that Classifier-Free Guidance (CFG) is critical for image quality. Our baseline model achieved an excellent FID of 4.90 using 20 inference steps and a 7.5 CFG scale, which surprisingly outperformed the 50-step model (FID 5.13).
- **Task 3 (Rectified Flow):** The "Reflow" technique was successfully implemented, enabling our model to produce images with an FID of 19.27 in just 5 inference steps. While this met the lab's objective (FID < 30), our specific model did not outperform our optimized Task 2 baseline, which achieved a better FID (7.12) [cite: 466] in the same 5 steps. Furthermore, we observed that the wall-clock time for Task 3 was nearly identical to Task 2 at the same step count.

### Future Work

Based on our findings, future work could proceed in several directions:

- **InstaFlow Implementation:** The most direct next step would be to implement the bonus Task 4 (InstaFlow). This would involve using our trained models to distill the multi-step generative process into a one-step model, completing the lab's exploration of sampling acceleration.

- **Optimizing Rectified Flow:** A deeper investigation is needed into why our Task 3 model did not surpass the Task 2 baseline. Future efforts could involve training the "Reflow" model for more epochs or exploring different methods for generating the  $(x_0, z_1)$  pairs to ensure a "straighter" path is learned.
- **Solver and Step Analysis:** Our finding that 20 steps outperformed 50 steps in Task 2 suggests a valuable area for further analysis. It would be beneficial to explore different ODE solvers (beyond simple Euler) or perform a more granular sweep of inference steps to better understand the trade-off between generative quality and computational cost.